

FDC3 Demo Playbook

Two prototype desktops · Sales walkthrough script with technical commentary

Opening framing

Both demos do the same thing: they show a working trader/banker desktop in two modes, side by side via a top-right toggle.

- **LEGACY · ANGULAR** — what a typical financial-services desktop looks like today: multiple apps stitched into one screen, but each one independent. The friction is real.
- **NEW · REACT + FDC3** — the same desktop with FDC3 wired in. One context, one channel, panels that talk to each other through a standard.

The data, the layout, the visuals are identical between modes. The only thing that changes is the wiring. That's the point — FDC3 isn't a UI library, it's an interop standard.

Desktop 1 — FX Sales Desk

*Open `built-standalone.html`. Default mode is **NEW · REACT + FDC3**, EUR/USD selected.*

Step 1 One click retunes the desk

Do Click **USD/JPY** in the watchlist (left column).

They see Five panels retune at once. Chart switches to USD/JPY with its own price series. RFQ Blotter filters to "USDJPY" — shows 0 requests · 1 total. Open Orders filters to USDJPY. News narrows from 9 to 4 / 9. The chart turns red because USD/JPY ticked down.

Underneath The watchlist called `fdc3.broadcast({ type: "fdc3.instrument", id: { ticker: "USDJPY" } })`. The other four panels each registered an `addContextListener("fdc3.instrument", ...)` at mount. The agent fanned the broadcast out to all of them on the current channel. **FDC3 context broadcast — the fundamental primitive.**

In Legacy mode, doing the same thing would mean changing the pair in five different dropdowns.

Step 2 Right-click → CreateOrder intent

Do Right-click any row in the watchlist (or the chart itself). Click **Order Entry**.

They see A floating orange-bordered Order Ticket window appears, locked to that pair, draggable. Pre-fills the side, notional, account. Submit it and you'll see it land in the Open Orders panel.

Underneath The watchlist called `fdc3.raiseIntent("CreateOrder", context)`. The agent looked at registered intent handlers, found one (the desktop shell), and routed it there. The shell opens the window. **FDC3 intent — verb-and-noun routing without panels knowing about each other.**

The same intent fires whether you right-click the chart or a watchlist row — different sources, same intent, same handler. The watchlist doesn't import the OrderTicket component. That decoupling is the whole point.

Step 3 News drives the chart via ViewChart intent

Do Right-click any news headline. Click **View Chart**.

They see The chart panel's border flashes orange briefly, and if the news is tagged with a different pair, the chart retunes to that pair. The rest of the desk follows.

Underneath News raised `fdc3.raiseIntent("ViewChart", { type: "fdc3.instrument", id: { ticker: "GBPUSD" } })`. The chart panel registered as the handler, and uses the visual pulse as a "I just got this" signal. **Intent + a side-effect of broadcasting context.**

Step 4 Channel switching

Do Top right, click any of the coloured dots in the channel picker (red is default).

They see All the panel headers' little channel dots change colour in unison. If you broadcast a different pair on a different channel, switching channels makes panels jump to whatever was last broadcast on that one.

Underneath `joinUserChannel("fdc3.channel.3")`. The agent stores last-context per channel per type, and replays it to listeners on the new channel. **FDC3 user channels — colour-coded workspaces, like Bloomberg.**

Step 5 Not everything needs to share context

Do Click between any pair in the watchlist. Watch the **LSEG Commodities Data** panel on the right.

They see Commodities (Oil Tanker Tracker, Gold Fixings, Natural Gas) **doesn't change**. It still shows its channel dot — it's joined the channel — but it ignores instrument broadcasts.

Underneath The commodities panel deliberately does NOT call `addContextListener("fdc3.instrument", ...)`.

FDC3 doesn't force coupling — apps opt in to listening per context type they care about. This is an important architectural nuance that often gets missed: FDC3 isn't "everything talks to everything", it's "things that should talk have a clean way to."

Step 6 Commodity chart modal (right-click → Open Chart)

Do Right-click any commodity row → **Open Chart**.

They see A draggable blue-bordered chart modal opens with that commodity's price history. Open multiple at once; they stack and drag independently.

Underneath Local UI concern, not an FDC3 intent — there's no `fdc3.commodity` context type, and Open Chart only opens this one panel. Worth pointing out: **the FDC3 standard ships ~30 context types and ~20 intents, but you can also use local plumbing when it fits.** Not everything has to be FDC3.

Step 7 Flip to Legacy mode

Do Top right, click **LEGACY · ANGULAR**.

They see The "● Apps share context via FDC3" tag turns into "● Apps operate independently". The channel picker disappears. Every panel now has its own dropdown for pair selection: chart, RFQ, orders, news, research — **five separate dropdowns**. Visual treatment goes greyscale.

Underneath Same panels, same data, agent installed but no one is broadcasting or listening. Switching the chart's dropdown doesn't affect anything else. To do what was one click in FDC3 mode, the salesperson now has to pick the pair five separate times.

This is the central architectural argument made visible in five seconds.

Desktop 2 — Wealth Manager Desktop

Open [built-wealth-standalone.html](#). Default mode is **NEW · REACT + FDC3**, Henderson Family Trust selected.

This is the more sophisticated demo. It shows **two FDC3 context types coexisting independently** on the same channel — `fdc3.contact` (the client) and `fdc3.instrument` (a holding within that client's portfolio).

Step 1 Click a client → six panels retune

Do Click **Voss Holdings LLC** in the Client Book.

They see Client Summary repopulates with Voss's details — UHNW, HIGH risk, Marcus Williams as RM, next review Jun 22. Portfolio Holdings shows Voss's 14 positions (tech-heavy: QQQ at top with 21% weight, NVDA, MSFT, AAPL, META, TSLA). Performance shows +17.3% vs +15.5% bench. Allocation donut redraws to Voss's mix. Activity narrows to Voss events: BUY 1,200 NVDA, SELL 400 TSLA, EMAIL "Hedging strategy discussion from M. Voss". Research narrows to 8/12. News narrows to 9/14.

Underneath Client Book broadcast { type: "fdc3.contact", id: { FDS_ID: "C002" }, name: "Voss Holdings LLC" }. Six panels registered `addContextListener("fdc3.contact", ...)` and re-rendered. **FDC3 context broadcast, primary context type for this desktop.**

Note that for wealth, the primary context is the **client**, not the instrument. That's a deliberate design decision — wealth desktops are client-shaped, not market-shaped.

Step 2 The architectural moment: a second context type

Do With Voss still selected, click **NVDA** inside the holdings table.

They see NVDA row highlights with orange accent. **Research narrows further: 8/12 → 3/12** (only NVDA-tagged research). **News narrows further: 9/14 → 2/14** (only NVDA headlines). **Activity stays on Voss** — does NOT change. **Client Summary stays on Voss** — does NOT change. **Performance & Allocation stay on Voss** — do NOT change.

Underneath The holdings table broadcast a *different* context type: { type: "fdc3.instrument", id: { ticker: "NVDA" } }. Research and News listen for *both* `fdc3.contact` AND `fdc3.instrument` and combine the filters. The other panels only listen to `fdc3.contact`, so they ignore the instrument broadcast.

This is the architectural point that's hard to make any other way. Two context types living on the same channel, neither one stomping the other. Each panel decides which types it cares about, and the rest is automatic.

The agent stores last-known-context-per-type-per-channel, not just last-known-context, so when a new listener joins later, it gets the correct latest contact AND the correct latest instrument — not just whichever was broadcast most recently.

Step 3 Right-click a client → StartCall intent

Do Right-click any client in the Client Book → **Schedule Call**.

They see A floating blue-bordered "Schedule Call" window opens, locked to that client. Pre-filled with the client name, call-type selector (Q Review / Onboarding / Tax Planning / Ad-hoc), date, time, attendees, agenda notes. Submit and it lands in the Activity panel.

Underneath `fdc3.raiseIntent("StartCall", contactContext)`. The desktop shell registered as the handler for StartCall and opens the window. **Same pattern as FX desk's CreateOrder — an FDC3 intent producing a floating, draggable, locked-context modal.**

Worth noting: StartCall is an actual FDC3 standard intent, not a made-up one. It's the same intent Symphony / Microsoft Teams integrations use — so in production, you could swap our mock window for a real telephony integration and **no panel code would need to change**. That's the value of using standard intents.

Step 4 Activity event drives the same intent

Do Right-click any row in the Activity panel → **Schedule Call**.

They see Same floating window opens, same client locked in.

Underneath Two different panels (Client Book and Activity) raise the same intent against the same context. The handler doesn't know or care which panel triggered it. **Decoupling between source and handler — classic intent pattern.**

Step 5 Drift indicator (quick callout)

Do Click between clients and point at the **Drift vs IPS** field in Client Summary.

They see Some clients show green "In line with IPS", others amber "Equity +3.2% vs target" or "Equity -4.1% vs target".

Underneath Not really an FDC3 feature — it's a derived analytic computed from the live portfolio composition vs the client's mandate. **Worth mentioning** because it's the kind of cross-data signal that's easy to add when all the data flows through a coherent context model. In the Legacy version, computing this would require manually re-querying the holdings and client metadata in every panel. Here it just falls out.

Step 6 Flip to Legacy mode

Do Top right, click **LEGACY · ANGULAR**.

They see Now the friction story for wealth becomes brutal. Six panels each gain their own client picker; three panels also gain a ticker picker. **Nine dropdowns total.** To switch to a different client, the wealth manager has to change six dropdowns. To then look at a specific holding's news and research, three more.

Underneath This is the real-world experience of a Goldman Wealth or BNY Wealth advisor today, using disconnected tools — and it's why FDC3 matters more in wealth than in any other vertical. Two clicks vs nine. Per workflow. All day.

Sales talking points

Memorise these.

On the standard itself

- FDC3 is a FINOS open standard, used in production by JPMorgan, Goldman, Citi, Barclays, Bank of America, Morgan Stanley.
- Version 2.2 (which these demos implement) ships ~30 standard context types and ~20 standard intents.
- Vendor-neutral: any FDC3-compliant container (OpenFin, Finsemble, Glue42, here:io) runs these apps unchanged.

On the migration path

- These demos use a custom in-browser agent (the prototype agent), but the *API surface the panels use is identical to the production standard*. Swap `installFdc3Agent()` for `await getAgent()` and the app runs on any compliant container with zero panel changes.
- That's the migration story: you build to FDC3, you don't lock in to a vendor.

On the "Angular → React" framing

- This isn't really about Angular vs React. It's about *uncoordinated apps vs interop-aware apps*.
- The Legacy mode in these demos could equally be old-React, jQuery, Knockout, server-rendered ASP.NET — anything siloed.
- The point isn't that Angular is bad, it's that any framework without an interop layer ends up with the same friction.

The pitch in one sentence

"We rebuild your panels in React — but more importantly, we wire them to FDC3 so they talk to each other, so they talk to your other vendors' apps, and so your salespeople do their job in one click instead of nine."

Quick reference — FDC3 primitives used

Each demo step maps to one of these standard FDC3 capabilities.

Demo step	FDC3 primitive	API call
Click watchlist / client / holding	Context broadcast	<code>fdc3.broadcast(context)</code>
Panel filters to current context	Context listener	<code>fdc3.addContextListener('fdc3.X', cb)</code>
Right-click → Order Entry / Schedule Call	Intent raise	<code>fdc3.raiseIntent('CreateOrder', ctx)</code>
Floating window appears	Intent listener	<code>fdc3.addIntentListener('CreateOrder', cb)</code>
Right-click news → View Chart, panel pulses	Intent + side-effect	<code>raiseIntent('ViewChart', ctx) → broadcast + pulse</code>
Top-right coloured-dot channel picker	User channels	<code>fdc3.joinUserChannel('fdc3.channel.3')</code>
Two context types coexist on same channel	Per-type context storage	<code>(agent-internal – last-context-per-type-per-channel)</code>

Single best moment to lean on

- If you're demoing to a CTO or architect: **Wealth Step 2** (two context types coexisting) — that's the architectural depth moment.
- If you're demoing to a sales head or business stakeholder: **either Legacy flip** — friction made visible in five seconds.
- If you have only 30 seconds: **FX Step 1** — one click, five panels retune. The immediate "ahh" moment.